CYBER SECURITY

Internal Assessment II

Implementation of a Security Project: Exploiting and Mitigating SQL Injection Vulnerabilities

(PortSwigger Web Security Academy)

N PAVITHRA
16010124182

1. Introduction

1.1 What is SQL Injection?

SQL Injection (SQLi) is a code-injection vulnerability class in which an attacker is able to interfere with the queries an application makes to its backend database. It arises when user-supplied input is concatenated directly into a SQL statement without proper sanitisation, validation, or parameterisation. Because the database trusts the final query string regardless of where its pieces came from, an attacker who controls part of that string can alter the query's logic — for example turning a single-row lookup into a query that returns every row in a table, or bypassing an authentication check entirely.

SQL injection has been a recognised vulnerability class since the late 1990s and has consistently ranked among the OWASP Top 10 most critical web application security risks (most recently under the broader "Injection" category, A03:2021). It remains common because it is simple to introduce — a single unsanitised string concatenation is enough — and because legacy codebases, rapid development cycles, and incomplete input validation continue to leave applications exposed.

1.2 Why It Matters

The impact of a successful SQL injection attack can range from unauthorised data disclosure to complete compromise of the underlying database and, in some configurations, the host operating system. Depending on the privileges of the database account used by the application, an attacker may be able to:

- Read sensitive data that the application was never designed to expose, such as other users' personal information, credentials, or financial records.
- Bypass authentication and authorisation controls, logging in as another user (including administrators) without knowing their password.

- Modify or delete data, undermining the integrity and availability of the application.
- In some cases, execute operating-system commands via the database server, leading to full server compromise.

Because the vulnerability sits at the intersection of application logic and data storage, a single flaw can expose an organisation's entire dataset at once, which is why SQL injection continues to be treated as a critical-severity issue in modern security assessments.

1.3 Purpose of This Report

This report documents a hands-on security project completed on the PortSwigger Web Security Academy, a free, legal training platform that provides deliberately vulnerable web applications for learning purposes. Four labs covering progressively more advanced SQL injection techniques were completed: exploiting a vulnerability in a WHERE clause to retrieve hidden data, bypassing a login form, and two UNION-based attacks used to extract data from other database tables. For each lab, this report records the objective, the methodology followed, supporting screenshot evidence, and what the result demonstrates about the underlying vulnerability. The report concludes with a discussion of practical mitigations and a real-world case study illustrating the consequences of this vulnerability class when left unaddressed in production.

2. Real-World Case Study: TalkTalk (2015)

In October 2015, UK telecommunications provider TalkTalk suffered a major data breach that is widely cited as one of the most significant real-world examples of the damage a SQL injection vulnerability can cause. Attackers exploited an SQL injection flaw in a webpage inherited from a company TalkTalk had acquired years earlier, which had not been properly decommissioned or patched.

2.1 What Happened

- Attackers used automated tools to probe TalkTalk's web infrastructure for SQL injection vulnerabilities and found an exploitable, internet-facing page.
- The injection allowed direct querying of the backend customer database, bypassing the application's intended access controls entirely.
- Personal data belonging to approximately 157,000 customers was accessed, including names, addresses, dates of birth, and email addresses; the bank account and sort code details of around 15,600 customers were also exposed.
- The attack was reportedly carried out largely by a group of young, relatively unsophisticated attackers, which illustrated how a low technical barrier combined with a known, patchable vulnerability class can still cause large-scale harm.

2.2 Consequences

- The UK Information Commissioner's Office (ICO) fined TalkTalk £400,000 — at the time the largest fine the ICO had issued — for failing to implement appropriate security measures.
- TalkTalk reported direct costs exceeding £40 million related to the incident, along with a substantial loss of customers and lasting reputational damage.
- The breach became a widely referenced case study in demonstrating that a vulnerability class as old and well-documented as SQL injection can still result in severe financial, regulatory, and reputational consequences when basic secure-coding and patch-management practices are not followed.

2.3 Relevance to This Project

The TalkTalk breach is a direct real-world illustration of the exact vulnerability class exercised in the labs documented in Section 4: an attacker-controlled input reaching a database query without adequate sanitisation. The labs in this report are deliberately simplified and run in a safe, legal training environment, but the underlying flaw — and its

potential consequences at scale — is the same one that led to TalkTalk's breach.

3–4. Lab Documentation

The four labs below were completed on the PortSwigger Web Security Academy. For each, the objective, methodology, supporting screenshots, and significance are documented.

Lab 1: SQL Injection Vulnerability in WHERE Clause Allowing Retrieval of Hidden Data

PortSwigger Web Security Academy — Lab category: SQL injection — retrieving hidden data

Objective

The goal of this lab was to exploit a SQL injection vulnerability in the product category filter of an e-commerce site in order to display products from a hidden category (one that is not normally shown to users, e.g. discontinued or unreleased items) that are not listed in the site's visible navigation.

Methodology

- Browsed the shop's category listing (e.g. "Corporate gifts") and observed how the selected category is passed as a parameter in the page URL (category=...).
- Identified that this parameter is used directly inside the application's backend WHERE clause without sanitisation.
- Modified the category parameter to append a boolean condition that always evaluates to true, causing the WHERE clause to match every row in the products table rather than only the selected category.
- Submitted the modified request and confirmed that products outside the normal category listing (including hidden/unreleased items) were returned.

Screenshots

Web Security Academy

SQL injection vulnerability allowing login bypass

Back to lab description »

LAB Not solved

Home | My account

WE LIKE TO SHOP







Web Security Academy

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Back to lab description »

LAB Solved

Congratulations, you solved the lab!

Share your skills!

Continue learning »

Home

WE LIKE TO SHOP

Corporate gifts' OR 1=1--

Refine your search:

All | Accessories | Clothing, shoes and accessories | Corporate gifts | Tech gifts

https://0acd00fc04c8a3688106af690093001e.web-security-academy.net/filter?category=Corporate gifts' OR 1=1--

What This Demonstrates

This lab demonstrates the most fundamental form of SQL injection: altering a query's WHERE clause so that it returns rows it was never intended to return. It shows how a single unsanitised parameter — here, a category filter that looks harmless on the surface — can be used to bypass the application's intended data-filtering logic entirely, exposing records the application developer explicitly did not want end users to see. This is the same underlying class of flaw referenced in the TalkTalk case study (Section 2), where an injected condition was used to retrieve data far beyond what any single legitimate query should return.

Lab 2: SQL Injection Vulnerability Allowing Login Bypass

PortSwigger Web Security Academy — Lab category: SQL injection — authentication bypass

Objective

The goal of this lab was to exploit a SQL injection vulnerability in the login function of the application in order to log in as the administrator user without knowing their password.

Methodology

- Inspected the login form and inferred that the username and password fields are likely concatenated directly into a SQL authentication query on the backend.
- Crafted a username value designed to terminate the intended WHERE clause early and comment out the remainder of the query — specifically the part of the query that checks the submitted password — so that the application's authentication check is satisfied without a valid password.
- Submitted the login form using the administrator username combined with the injected comment sequence, leaving the password field value irrelevant to the outcome.
- Verified that the application logged the session in as the administrator account and granted access to the administrator's account page.

Screenshots

Lab: SQL injection vulnerability allowing login bypass

APPRENTICE

LAB | Not solved

This lab contains a SQL injection vulnerability in the login function.

To solve the lab, perform a SQL injection attack that logs in to the application as the `administrator` user.

ACCESS THE LAB

Web Security Academy | SQL injection vulnerability allowing login bypass

Back to lab description »

LAB Not solved

Home | My account

WE LIKE TO SHOP



Robot Home Security Buddy

$96.48



WTF? - The adult party game

$51.72



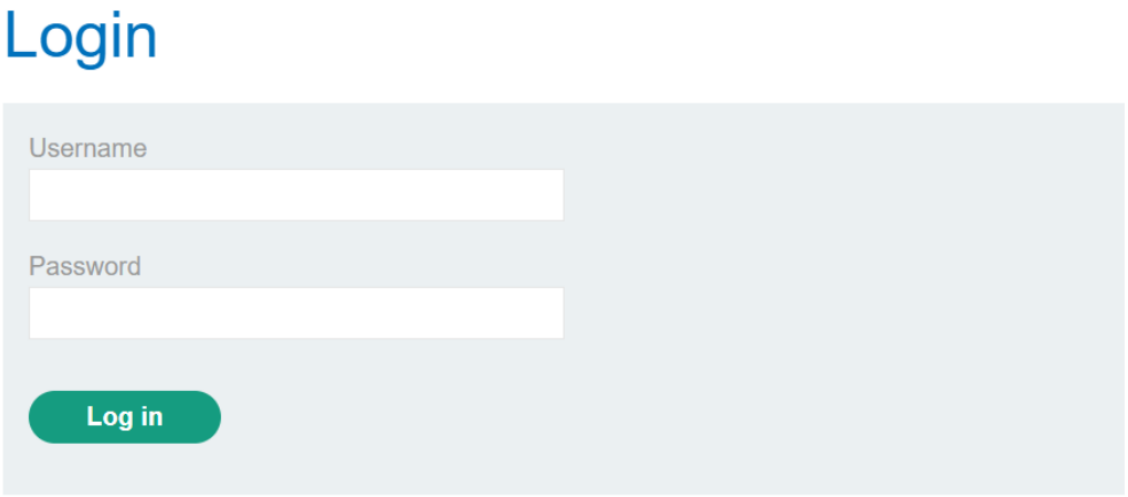
The Lazy Dog

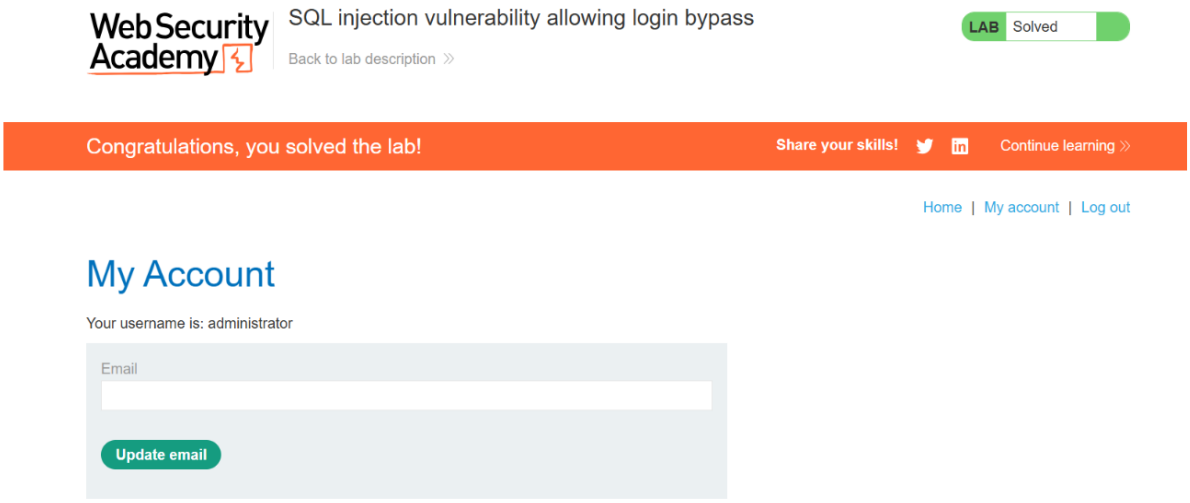
$30.01



Folding Gadgets

$31.66





What This Demonstrates

This lab demonstrates how SQL injection can defeat authentication logic directly, rather than only being used to read data. Because the login query itself is built by string concatenation, an attacker can restructure the query's logic so that the password comparison is never evaluated — effectively convincing the application that a correct password was supplied. This highlights why authentication code is considered especially high-risk: a single injection flaw here does not just leak data, it can hand an attacker a fully authenticated, privileged session.

Lab 3: SQL Injection UNION Attack — Determining the Number of Columns Returned by the Query

PortSwigger Web Security Academy — Lab category: SQL injection — UNION-based data extraction

Objective

The goal of this lab was to determine the number of columns returned by the underlying SQL query, which is a required first step before a UNION-based injection can be used to extract data from other tables.

Methodology

- Started from the category filter parameter identified as injectable in Lab 1.
- Submitted a `UNION SELECT NULL--` payload in the category parameter and observed the response.
- On receiving an "Internal Server Error," increased the number of `NULL` values by one (`NULL,NULL--`, then `NULL,NULL,NULL--`) and resubmitted each time.
- Continued until the request returned no error and the page loaded normally, confirming the query returns exactly that many columns.
- Confirmed the lab flipped to "Solved" once the correct number of `NULL`s was used.

Screenshots

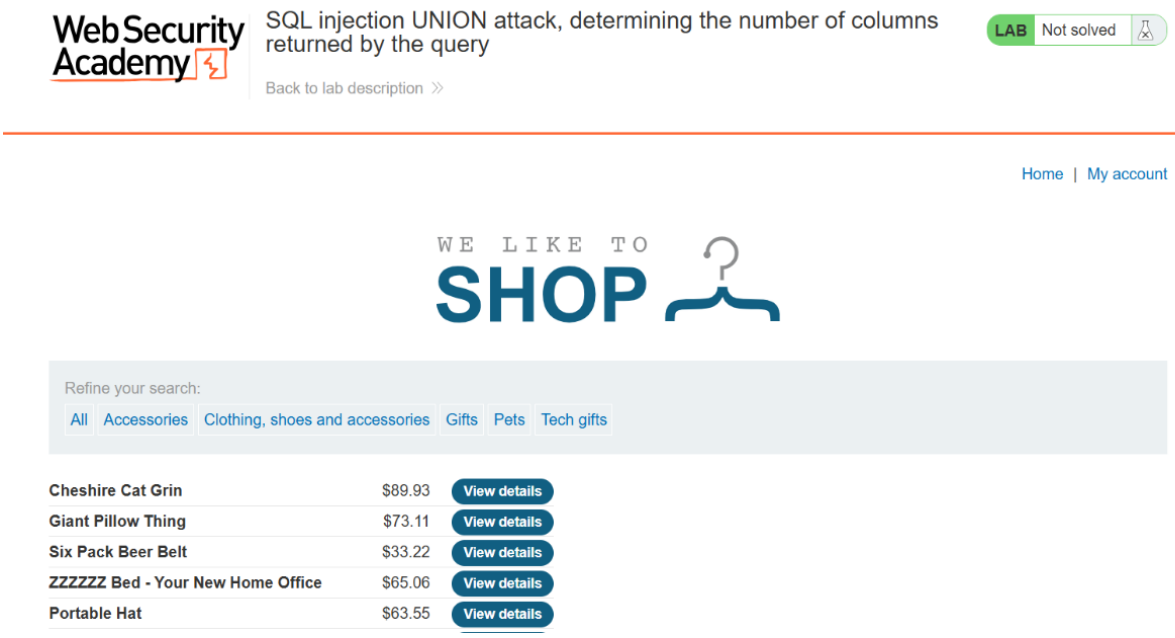
Lab: SQL injection UNION attack, determining the number of columns returned by the query

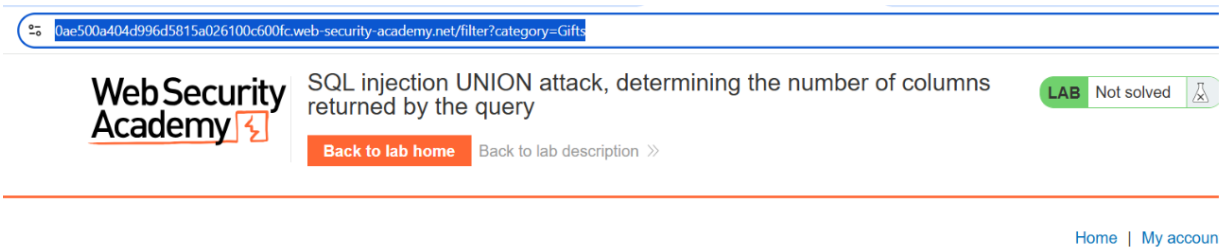
PRACTITIONER

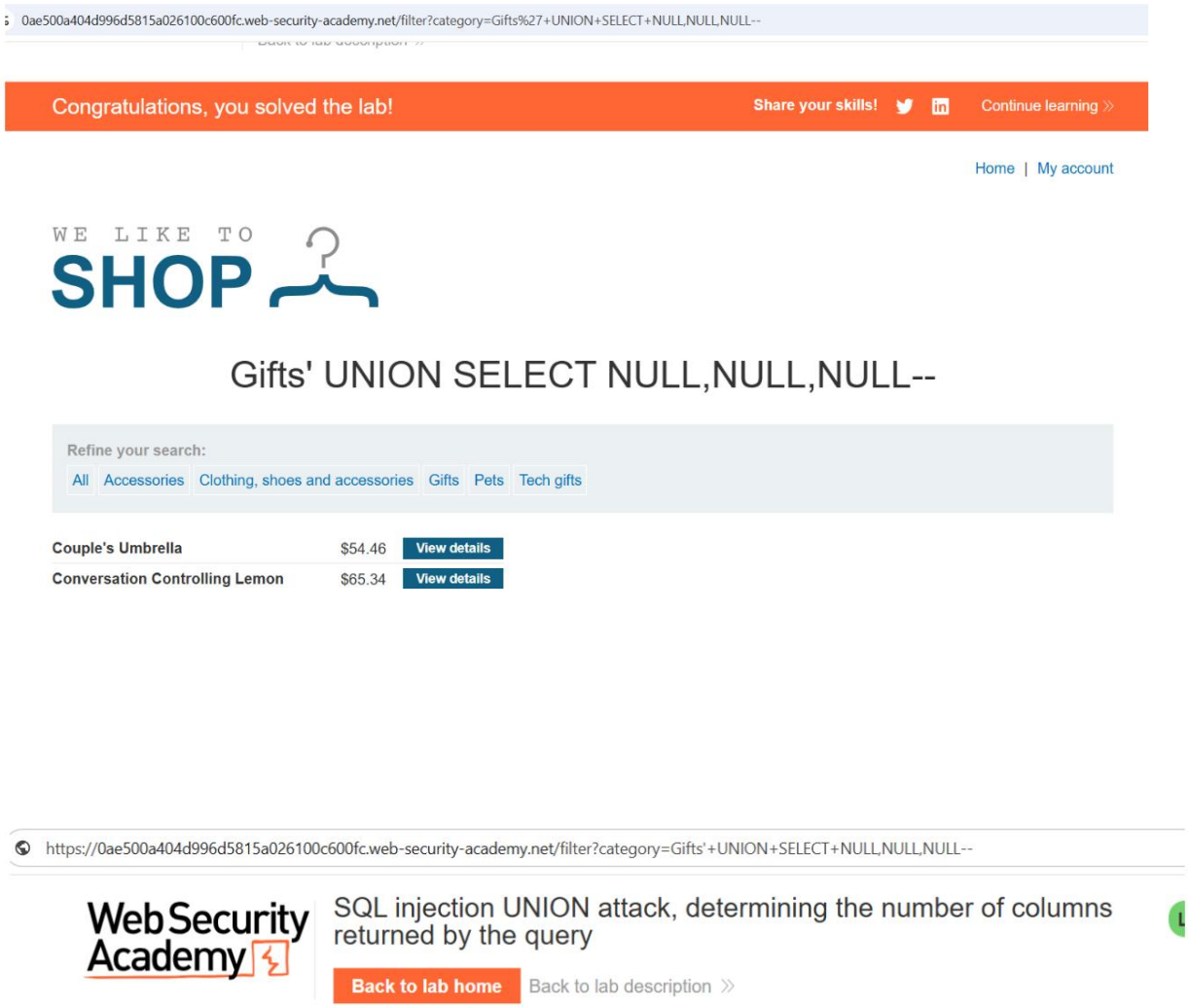
LAB Not solved

This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response, so you can use a UNION attack to retrieve data from other tables. The first step of such an attack is to determine the number of columns that are being returned by the query. You will then use this technique in subsequent labs to construct the full attack.

To solve the lab, determine the number of columns returned by the query by performing a SQL injection UNION attack that returns an additional row containing null values.







What This Demonstrates

This lab demonstrates the reconnaissance step that precedes any UNION-based SQL injection attack. Before an attacker can use UNION SELECT to pull arbitrary data from other tables, the injected query must return the same number of columns as the original query, or the database will reject it with an error. By systematically probing with ORDER BY and UNION SELECT NULL, the exact column count can be inferred purely from the application's success/error responses — without needing any direct error-message disclosure. This is a reminder that even generic, non-descriptive

error responses can leak structural information about a backend database if input validation is absent at the parameter level.

Lab 4: SQL Injection UNION Attack — Finding a Column Containing Text

PortSwigger Web Security Academy — Lab category: SQL injection — UNION-based data extraction

Objective

Building on Lab 3, the goal of this lab was to identify which of the query's columns could hold string (text) data, so that it could later be used to display arbitrary extracted data — such as values from other tables — back on the page.

Methodology

- Used the column count established in Lab 3 as the number of values in a UNION SELECT statement.
- Replaced the NULL placeholders one at a time with a short, recognisable text string, submitting a separate request for each column position.
- Observed the application's response for each attempt: a database type-mismatch error indicates that column does not accept string data, while a successful response containing the injected string confirms the column can hold and display text.
- Identified the specific column position where the injected string was returned, which is the position that could be used to exfiltrate arbitrary text-based data from other database tables.

Screenshots

Lab: SQL injection UNION attack, finding a column containing text

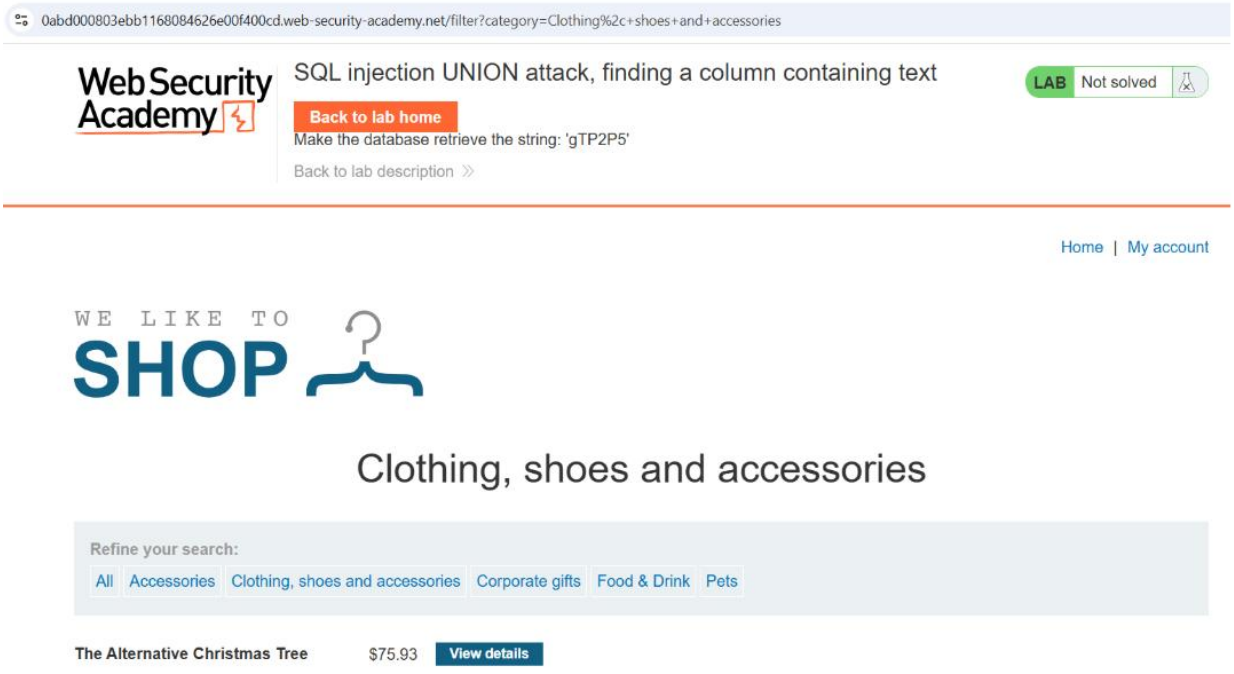
PRACTITIONER

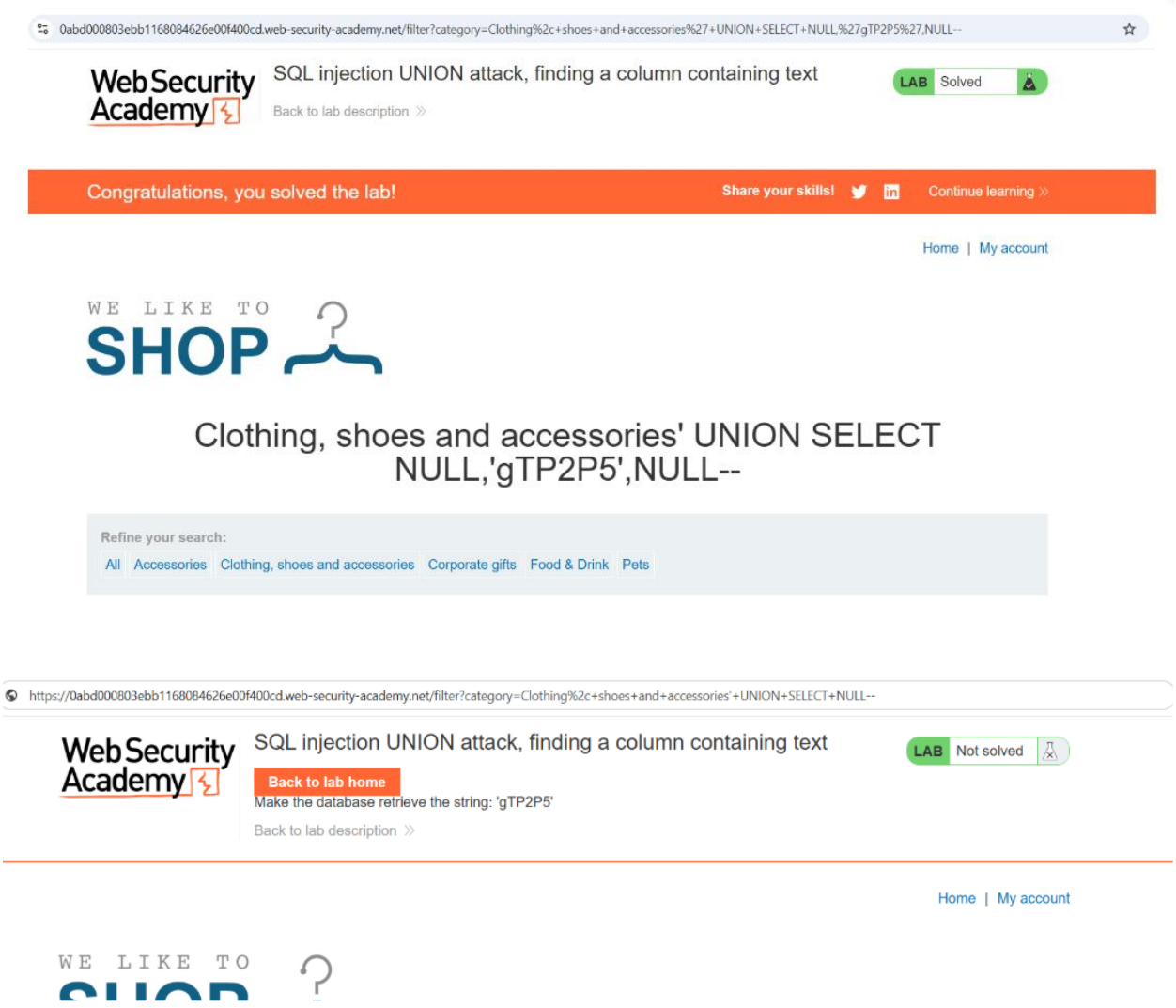
LAB Not solved

This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response, so you can use a UNION attack to retrieve data from other tables. To construct such an attack, you first need to determine the number of columns returned by the query. You can do this using a technique you learned in a previous lab. The next step is to identify a column that is compatible with string data.

The lab will provide a random value that you need to make appear within the query results. To solve the lab, perform a SQL injection UNION attack that returns an additional row containing the value provided. This technique helps you determine which columns are compatible with string data.

ACCESS THE LAB





What This Demonstrates

This lab completes the groundwork needed for a full UNION-based data-extraction attack. Once an attacker knows both the number of columns (Lab 3) and which of those columns can carry string data (Lab 4), they are in a position to substitute that column with a query against any other accessible table — for example, a users table holding usernames and password hashes — and have the result rendered directly on the page. This two-step process (count columns, then find a text-compatible column) is the standard methodology behind nearly all UNION-based SQL injection attacks, and mirrors the technique that would be used to extract the kind

of customer data exposed in incidents such as the TalkTalk breach discussed in Section 2.

5. Mitigations

Each of the four labs in this report exploited the same root cause: untrusted input being incorporated into a SQL query without proper separation between code and data. The following mitigations address this root cause and are considered current industry best practice.

5.1 Parameterised Queries (Prepared Statements)

The single most effective defence against SQL injection is the use of parameterised queries, where the SQL query structure is defined separately from the user-supplied values. The database driver treats placeholders strictly as data, never as executable SQL syntax, which means that injected characters such as quotes or comment sequences cannot alter the query's logic — regardless of what the user submits. This directly prevents every technique demonstrated in Labs 1 through 4, since the injected conditions, comment sequences, and UNION clauses used in this report rely entirely on the application treating user input as part of the SQL syntax itself.

5.2 Input Validation and Allow-Listing

Where user input is expected to come from a known, limited set of values (such as a category filter), the application should validate that input against an allow-list of acceptable values rather than passing it through to the query unmodified. Generic input validation — rejecting unexpected characters such as quotes, semicolons, or SQL keywords — provides a secondary layer of defence, though it should never be relied on as the sole protection, since allow-list and block-list filtering can often be bypassed through encoding or alternative syntax.

5.3 Principle of Least Privilege for Database Accounts

The database account used by an application should be granted only the minimum privileges required for that application's normal functioning — for example, read-only access to the specific tables it needs, with no permission to access unrelated tables, execute administrative commands, or interact with the underlying operating system. Had such least-privilege

controls been enforced, the impact of a successful injection (such as the UNION-based extraction demonstrated in Labs 3 and 4) would have been limited to the explicitly permitted tables, rather than the entire database.

5.4 Additional Defence-in-Depth Measures

- Use of a Web Application Firewall (WAF) to detect and block common injection patterns at the network edge, as an additional layer rather than a replacement for secure coding.
- Regular use of static and dynamic application security testing (SAST/DAST) tools during development and before deployment to catch injectable parameters early.
- Centralised, consistent use of an Object-Relational Mapping (ORM) framework, which parameterises queries by default and reduces the chance of a developer manually concatenating SQL strings.
- Routine patching and decommissioning of legacy or inherited systems — the TalkTalk breach (Section 2) was ultimately traced to a page that should have been retired rather than left exposed.
- Generic, non-descriptive error messages returned to the end user, so that database structure (table names, column counts, or error syntax) cannot be inferred from application responses, as was exploited during reconnaissance in Lab 3.

6. Conclusion

This project provided hands-on experience with four progressively advanced SQL injection techniques, carried out in a safe and legal training environment provided by the PortSwigger Web Security Academy. Starting from a basic WHERE-clause manipulation used to retrieve hidden data, the project progressed through an authentication-bypass attack and then through the two-stage reconnaissance process (column-count enumeration followed by column-type identification) that underpins UNION-based data extraction.

Across all four labs, the underlying cause was consistent: user-controllable input reaching a SQL query without adequate separation between code and data. The TalkTalk breach of 2015 illustrates that this is not merely an academic concern — the same class of vulnerability, left unaddressed in a production system, resulted in the exposure of over 150,000 customers' personal data, a regulatory fine, and tens of millions of pounds in direct costs.

The mitigations discussed in Section 5 — most importantly, the consistent use of parameterised queries, combined with input validation, least-privilege database accounts, and defence-in-depth measures such as WAFs and routine security testing — directly address the root cause exploited in each of these labs. Understanding both the offensive technique and the corresponding defence is essential for building and maintaining secure applications.

7. References

- PortSwigger Web Security Academy. "SQL injection." https://portswigger.net/web-security/sql-injection
- PortSwigger Web Security Academy. "Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data." https://portswigger.net/web-security/sql-injection/lab-where-clause-retrieve-hidden-data
- PortSwigger Web Security Academy. "Lab: SQL injection vulnerability allowing login bypass." https://portswigger.net/web-security/sql-injection/lab-login-bypass
- PortSwigger Web Security Academy. "Lab: SQL injection UNION attack, determining the number of columns returned by the query." https://portswigger.net/web-security/sql-injection/union-attacks/lab-determine-number-of-columns
- PortSwigger Web Security Academy. "Lab: SQL injection UNION attack, finding a column containing text." https://portswigger.net/web-security/sql-injection/union-attacks/lab-find-column-containing-text

- OWASP Foundation. "OWASP Top 10:2021 – A03:2021 Injection." https://owasp.org/Top10/A03_2021-Injection/
- OWASP Foundation. "SQL Injection Prevention Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- Information Commissioner's Office (ICO). "TalkTalk gets record £400,000 fine for failing to prevent October 2015 attack." ICO Press Release, 2016. https://ico.org.uk/
- BBC News. "TalkTalk hack 'affected 157,000 customers'." 6 November 2015. https://www.bbc.com/news/business-34743185